3. Array Implementation of List ADT

T.V. GEETHA

epgp books

Welcome to the e-PG Pathshala Lecture Series on Data Structures.This is the third module of the paper and we will be talking about the array implementation of List ADT.

Learning Objectives

The learning objectives of this module are as follows:

- To discuss the different implementations of List ADT
- To understand the array and array based vector implementation of Lists
- To outline the implementation of Lists using dynamic arrays

3.1 Implementation of an ADT

Before we go on to describe the implementation of a particular type of ADT let us discuss the general issues in implementing an ADT. The first step in the implementation is the choiceof the data structure to represent the ADT's data. This choice of the data structure depends mainly on details of the ADT's operations and the context in which the operations will be used. You must remember that the implementation details should be hidden behind a wall of ADT operations. In other words a program would only be able to access the data structure using the ADT operations.

The first step in implementing an ADT is choosing a **data structure**such as arrays, records, etc. to represent the ADT. Each operation associated with the ADT is implemented by one or more subroutines. Two standard implementations for the list ADT that we will be discussing in this paper are array-based implementation and linked list based implementation. In this module we will be discussing the array based implementation of the List ADT.

3.2 Static Implementation of ADT List

The simplest method to implement a List ADT is to use an array that is a"linear list" or a "contiguous list" where elements are stored in contiguous array positions. The implementation specifies an array of a particular maximum length, and all storage is allocated before run-time. It is a sequence of n-elements where the items in the array are stored with the index of the array related to the position of the item in the list.

In array implementation,elements are stored in contiguous array positions (Figure 3.1). An array is a viable choice for storing list elements when the elements are sequential, because it is a commonly available data type and in general algorithm development is easy.

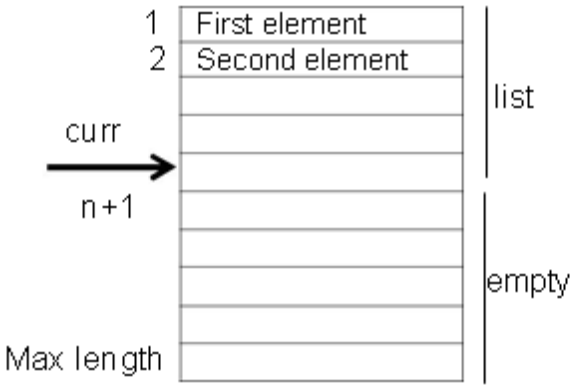

Figure 3.1 Array Implementation of List ADT

3.2.1List Implementation using arrays

In order to implement lists using arrays we need an array for storing entries – **listArray[0,1,2…..m],** a variable **curr**to keep track of the number of array elements currently assigned to the list, thenumber of items in the list, or current size of the list **size**and a variable to record the maximum length of the array, and therefore of the list – **maxsize as shown in Figure 3.2.**

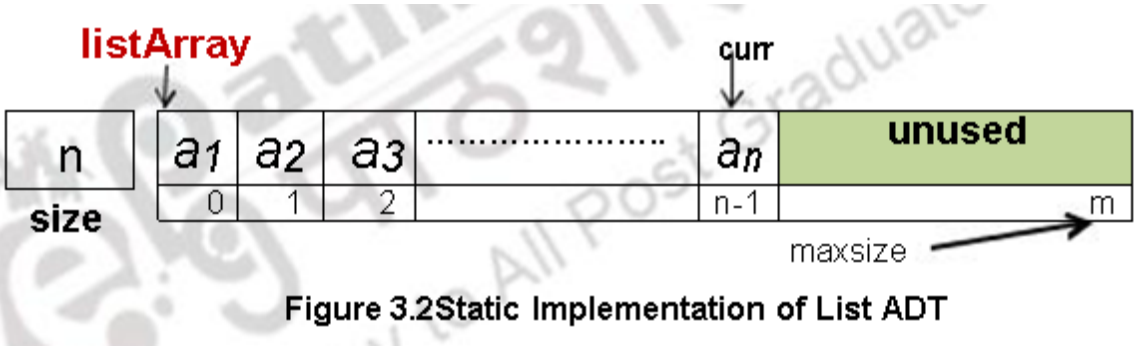

Figure 3.2Static Implementation of List ADT

Fix one end of the list at index 0 and elements will be shifted***as needed*** when an element is added or removed. Therefore insert and delete operations will take O(n). That is if we want to insert at position 0 (insert as first element),this requires first pushing the entire array to the right one spot to make room for the new element. Similarly deleting the element at position 0requires shifting all the elements in the list left one spot.

One of the important operations associated with lists is the initialization. In this case we need to first declare the array of list items with maximum number of items say n. This then requires an estimate of the maximum size of the list. Once we decide the size, then we `initialize the list with number of items set at 0.

3.2.1.1 Finding First Element

Here we define finding the first element by first calling a Boolean function – IsEmpty list. If IsEmpty listis true that is if the list is empty then there is no question of finding an element and finding the first element will return a value that is not a valid entry of the list However if IsEmpty listis false that is the list is not empty the element at the first location position (0) is returned.

3.2.1.2 Insertion

What happens if you want to insert an item at a specified position in an existing array?The item originally at the given index must be moved right one position, and all the items after that index must be ***shifted right***(Figure 3.3).

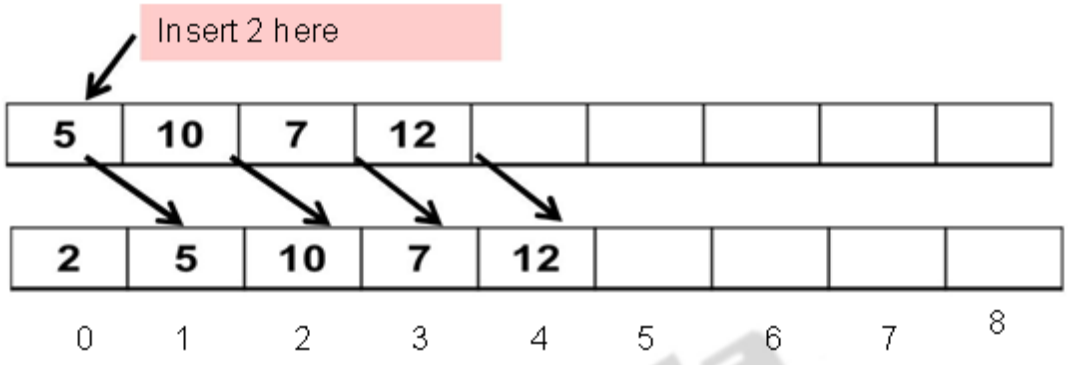

Figure 3.3 Insertion into a List

Let us consider a specific case of Insert operation. Insert (3, K, List) In this case, 3 is the index or the point of Insertion, K is the element to be inserted & List is the list in which insertion is to be done (Figure 3.4 (a) & Figure 3.4 (b)).

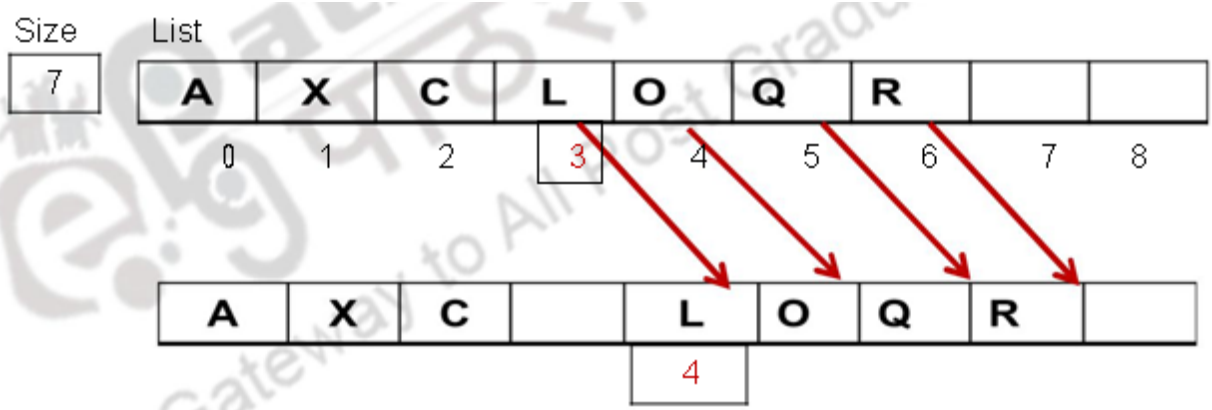

Figure 3.4 (a)An Example of Insertion into a List

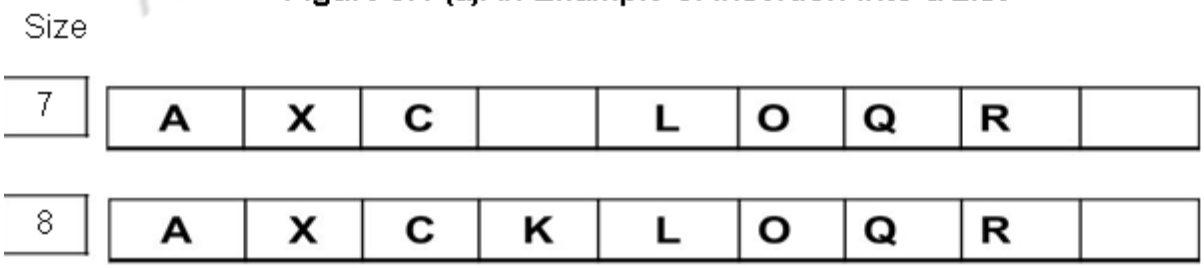

Figure 3.4 (b)An Example of Insertion into a List

Let us see the steps in the insertion with the example given in Figure 3.4(a) & Figure 3.4 (b). Let us assume initially that the original list contains 7 elements (that is Size =7) namely A at array location 0, X at location 1 and so on and finally Rat location 6. The steps involved are as follows:

Step 1: First check whether there is place to add the new element – in other words whether the array is full. If so indicate error

size = maxsize – Then error

Step 2: Now we need to make room for the new element at the index position 3. We need to shift elements from index to curr (curr position of last element of list) one position to the right.

Loop till curr
Items[index+1] =>tems[index]

Step 3: Write the element into the gap created by shifting the elements to the right

Items[index] => K

Step 4: Update size and curr position of the list
size=> size +1; curr=>curr +1

3.2.2.2 Deletion from Lists

What happens if you want to remove an item from a specified position in an existing array? There are two ways in which this can be done:

– Leave ***gaps*** in the array, i.e. indices that contain no elements, which in practice, means that the array element has to be given a special value to indicate that it is "empty", or

– All the items after the (removed item's) index must be ***shifted left***similar to what we did when we wanted to insert – only for insertion we shifted right.

– **Delete (3, List)**. In this case, 3 is the index or the point of Deletion& List is the list from which deletion is to be done (Figure 3.5 (a) & Figure 3.5 (b)).

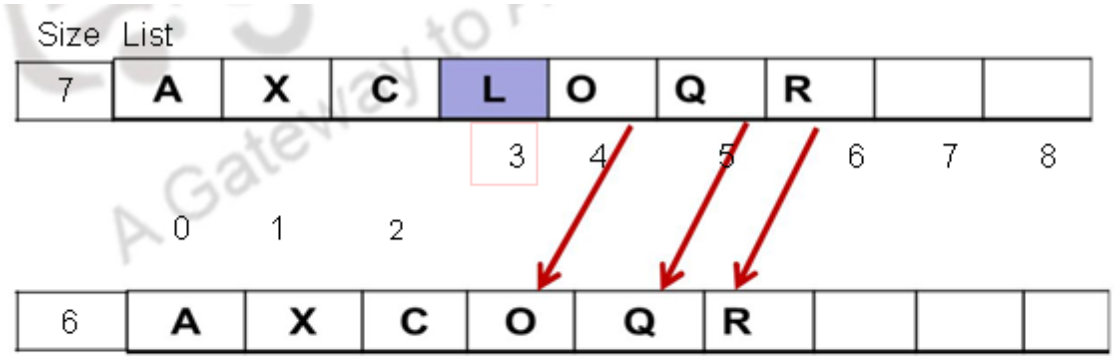

Figure 3.5 (a) Deletion from a List

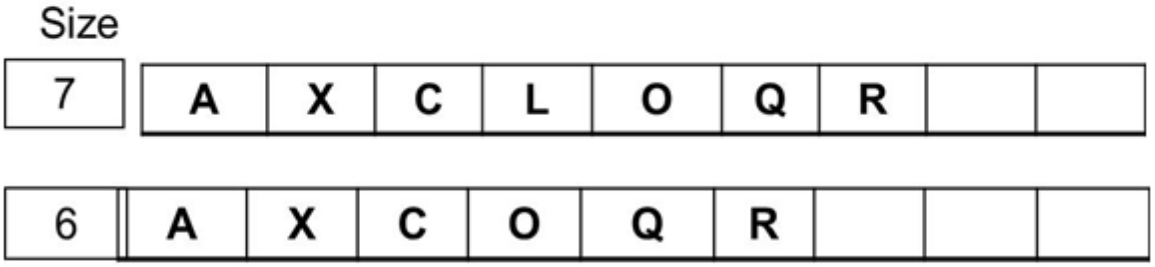

Figure 3.5 (b) Deletion from a List

Let us see the steps in the deletion with the example given in Figure 3.5(a) & Figure 3.5(b). Let us assume initially that the original list contains 7 elements (that is Size

=7) namely A at array location 0, X at location 1 and so on and finally R at location 6. The steps involved are as follows:

Step 1: First check whether the list is empty that is whether there are elements in the list.

We cannot perform deletion from an empty list.

size=0 Then error

Step 2: Now we need to shift elements from index position 4 to the left so that an empty locationis created due to the deletion at location 3 will be filled by elements to the left. We need to shift elements from index (i) to curr(that is the curr position of last element of list) one position to the left.

Loop till curr

Items[index] => items[index+1]

Step 4: Update size and curr position of the list

size=>size-1; curr=>curr-1

3.2 Advantages of Array-Based Implementation of Lists

Some of the major advantages of using array implementation of lists are:

- Array is a natural way to implement lists
- Arrays allow fast, random access of elements
- Array based implementation is memory efficient since very little memory is required other than that needed to store the actual contents

Some of the disadvantages of using arrays to implement lists are:

- The size of the list must be known when the array is created and is fixed (static)
- Array implementations of lists use a static data structure. Often defined at compile-time. This means the array size or structure cannot be altered while program is running. This requires an accurate estimate of the size of the array.
- This fixing of the size beforehand usually results in overestimation of size which means we tend to usually waste space rather than have program run out.

- The deletion and insertion of elements into the list is slow since it involves shifting of elements. It also means that data must be added to the end of the list for insertion and deletion to be efficient. If insertion and deletion is towards the front of the list, all other elements must shuffle down. This is slow and inefficient. This inefficiency is even more pronounced when the size of the list is large.

3.3 The Vector ADT

The Vector ADT extends the notion of array by storing a sequence of arbitrary objects. However instead of using position or index to access an element, an element can be accessed, inserted or removed by specifying its rank that is the number of elements preceding it. An exception will be thrown if an incorrect rank is specified (e.g. negative rank, or larger than current size). The main vector based list operations are as follows:

- elemAtRank(integer r): This operation returns the element at rank r without removing it.
- replaceAtRank(integer r, object o): This operation replaces the element at rank with an element o and returns the old element.
- insertAtRank(integer r, object o): This operation inserts a new element o so that it has rank r
- removeAtRank(integer r): This operation removes and returns the element at rank r Other common operations includesize() and isEmpty().

3.3.1 Applications of Vectors

Vectors have some specific applications. Direct applications include sorted collection of objects like in an elementary database. There are also some indirect applications such as use as an auxiliary data structure for algorithms and as components of other data structures.

3.3.2 Array-based Vector

In this implementation we use an array **V**containing ***n****elements.* A variable ***size=n*** is used to keep track of the size of the vector (number of elements stored). Normally an array is addressed by its index and when representing the list by an array the

rank and index are identical. However we differentiate the vector representation with rank to explain the important operations to be supported when using a vector.

The important operation ***elemAtRank*(*r*)** is implemented in **O**(1) time by returning **V**[***r***] (Figure 3.6). Similar to array implementation of lists, in operation ***insertAtRank*(*r*, *o*)**, we need tomake room for the new element by shifting forward the $n - r$ elements **V**[***r***], ...,**V**[n-1] (Figure 3.7). In the worst case (r =0), this takes **O**(m) time

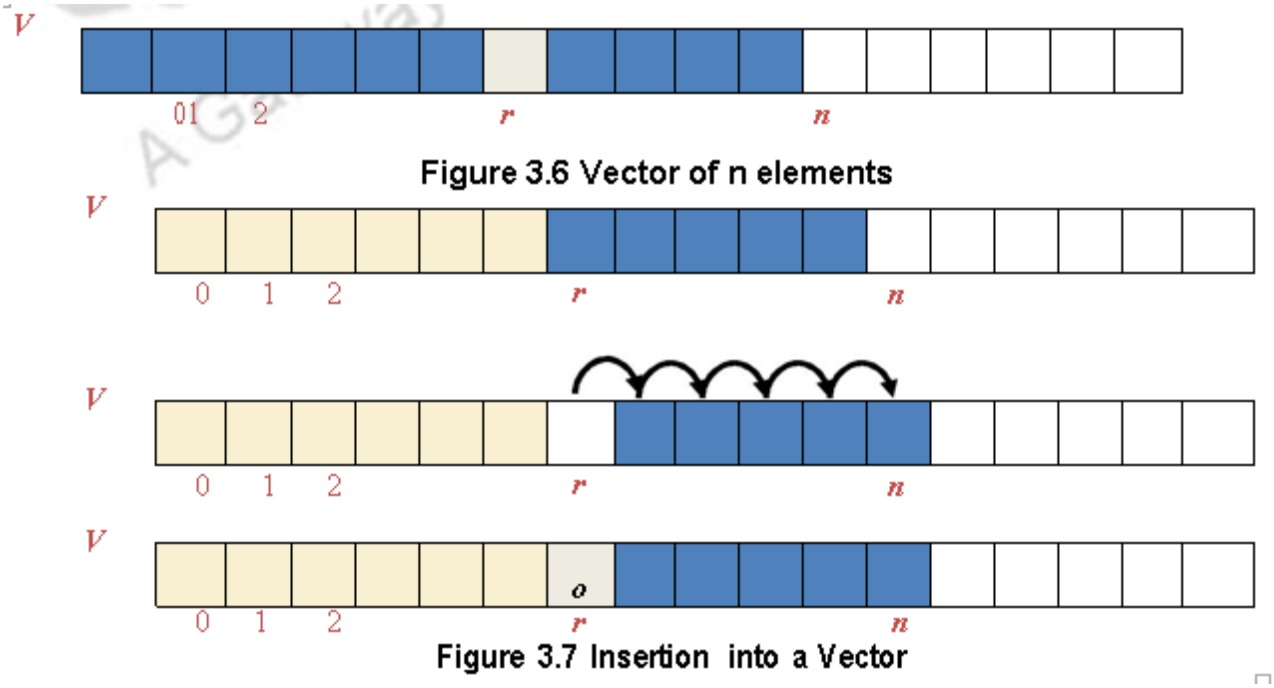

Figure 3.6 Vector of n elements

Figure 3.7 Insertion into a Vector

Similar to array implementation of lists, in operation ***removeAtRank*(*r*)**, we need to fill the hole left by the removed element by shifting backward the $n - r$ -1 elements **V**[r +1], ...,**V**[n -1] (Figure 3.8). Again in the worst case (r =0), this takes **O**(n) time.

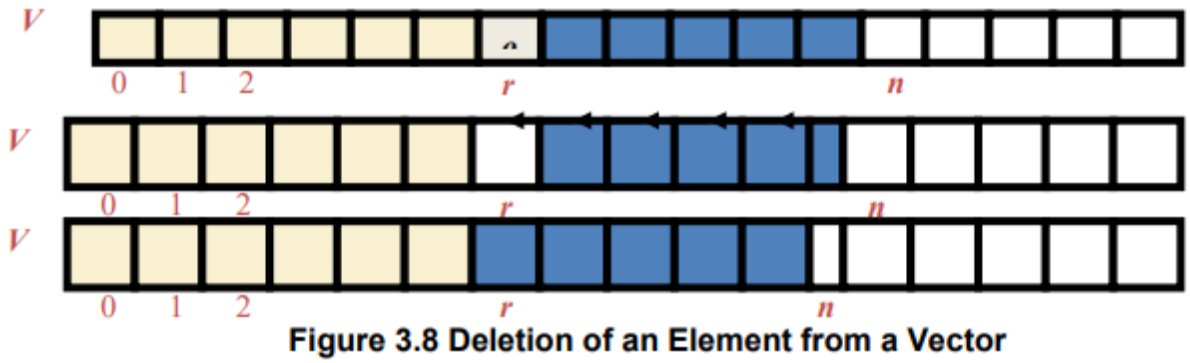

Figure 3.8 Deletion of an Element from a Vector

3.3 Performance

In the array based implementation of a Vector, the space used by the data structure is **O**(***n***). The operations ***size***, ***isEmpty***, ***elemAtRank*** and ***replaceAtRank***all run in **O**(1) time. However the operations ***insertAtRank*** and ***removeAtRank*** run in **O**(***n***) time. If we use the array in a circular fashion,***insertAtRank***(0) and ***removeAtRank***(0) run in **O**(1) time. In an ***insertAtRank*** operation, when the array is full, instead of throwing an exception, we can replace the array with a larger one

3.4 ADT List using Dynamic arrays

A dynamic data structure is one that changes size, as needed, as items are inserted or removed. There is usually no limit on the size of such structures, other than the size of main memory. Dynamic arrays are arrays that grow (or shrink) as required. In other words a new array is created when the old array becomes full by creating a new array object, copying the values from the old array and then assigning the new array to the existing array reference variable

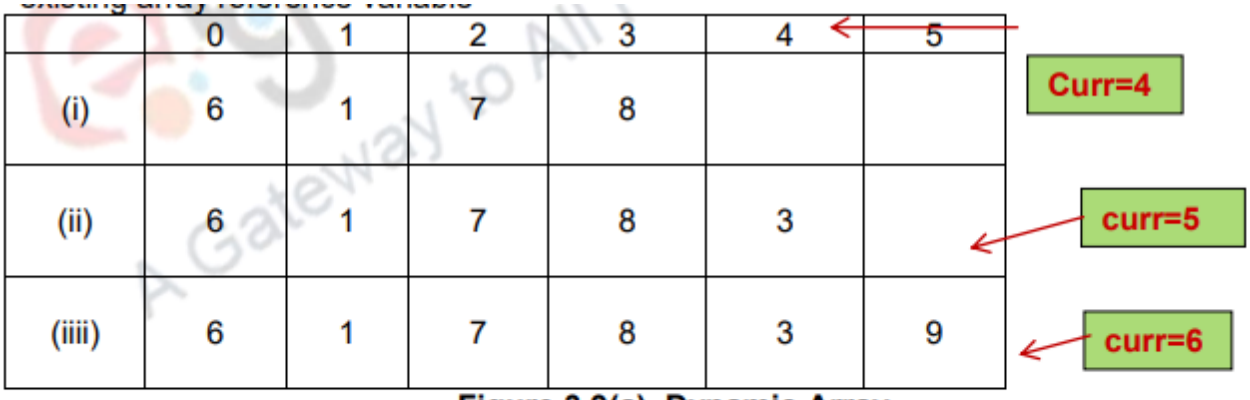

Figure 3.9(a) Dynamic Array

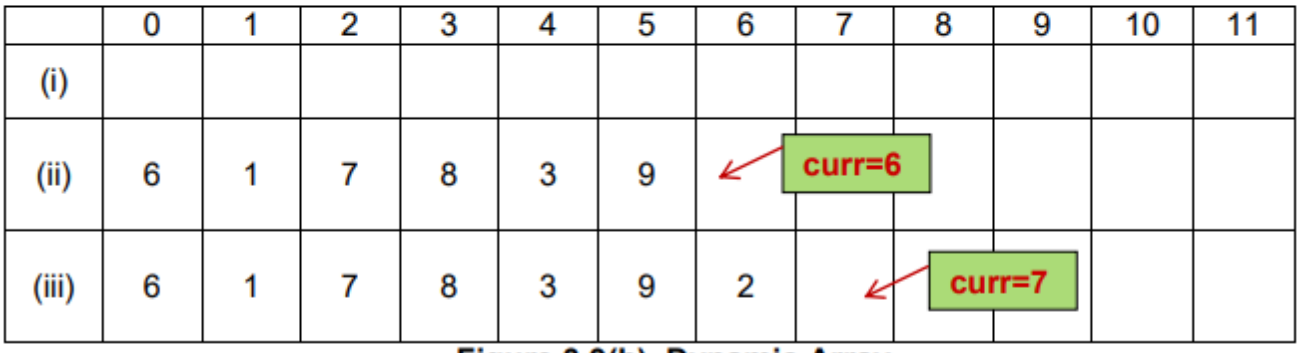

Figure 3.9(b) Dynamic Array

Let us see the steps in the insertion of an element into a dynamic array with the example given in Figure 3.9(a) & Figure 3.9(b). Let us assume initially that the original array has a size of 6 (0,1 ….5) and contains 4 elements **6,1,7,8** occupying locations 0,1,2 and 3 respectively, with curr pointing to 4 the next empty location (Figure 3.9(a) – (i)). The steps involved are as follows:

Step 1: Now let us assume that we want to insert **3**. Now element 3 will be inserted in location 4 and curr will be incremented to 5 (Figure 3.9(a) – (ii)).

Step 2:Now let us assume that we want to insert **9**. Now element 9 will be inserted in location 5 and curr will be incremented to 6 (Figure 3.9(a) – (iii)).

Step 3: Now let us assume that we want to insert **2**. At this point curr>maxsize of array and list full condition will be signaled as there is no room for a new item. In static array – an exception will arise.

Step 4: However in the case of dynamic arrays, we will create a new, bigger array as shown in Figure 3.9(b) – (i).

Step 5: Now all the elements of the old array will be copied to the new array and curr set to 6 (Figure 3.9(b) – (ii)).

Step 6: Now finally we can insert 2 into the new array and accordingly curr is set to 7 (Figure 3.9(b) – (iii)).

The old array will eventually be deleted

3.4.2 Problems associated with Dynamic Array

Insertion using dynamic array essentially means that before every insertion, we need to check if the array needs to grow. An important question that arises is that when growing an array, how much to grow it?. Is this decision based on memory efficiency that is grow by one only – since we need one location when one element is to be inserted? But in this case the time efficiency is very poor since the expensive copy operation has to be carried out every time an insertion occurs. Growing by doubling works well in practice, because it grows very large very quickly -10, 20, 40, 80, 160, 320, 640, 1280,…and so on.In this case very few array re-

sizingsare needed. While the copying operation in this case is again expensive, this copying does not have to be done often.

When the doubling does happen it may be time-consumingbutright after a doubling half the array is empty. However re-sizing after each insertion would be prohibitively slow. Deleting and inserting in the middle of an array (on average) is still of the order of O(n).

3.4 Summary

- Explained the different implementations of Lists using arrays
- Discussed the Pros and Cons of each implementation
- Discussed the different operations possible with array based implementation of Lists

---

you can view video on Array Implementation of List ADT 

Web Links

- en.wikipedia.org/wiki/List_(abstract_data_type)
- www.cs.cmu.edu/~tcortina/15110sp12/Unit06PtA.pdf
- www.csee.umbc.edu/courses/undergraduate/341/.../Lists/List1.html
- www.doc.ic.ac.uk/~ar3/lectures/ProgrammingII/.../Lecture2PrintOut.pdf
- www.cs.sfu.ca/CourseCentral/225/...notes/dynamic-arrays-link_list.pdf
- www.math.bas.bg/~nkirov/2009/NETB201/slides/ch05/ch05.html
- www.cs.sfu.ca/CourseCentral/225/amhunter/lecturenotes/DynamicArray.ppt
- http://orion.lcg.ufrj.br/Dr.Dobbs/books/book3/chap3.htm
- cs.txstate.edu/~rp44/cs3358_092/Lectures/list_revised.ppt

Supporting & Reference Materials

1. Carrano and Henry, “Data Structures and Problem Solving with C++: Walls and Mirrors”, Pearson; 6 edition, 2012
2. Mark Allen Weiss, “Data Structures and Algorithm Analysis in Java”, Pearson; 3rd Edition, 2011

3. Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, “Data Structures and Algorithms in Java”, Wiley; 6 edition, 2014